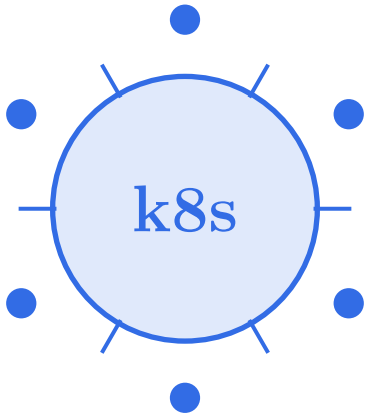
Kubernetes mel Sifr

Guide Complet bel Darija Tounsia



N3almou Kubernetes
bel projet mte3na ka example pratique

Vagrant · Ansible · containerd · Flannel · Gitea · Nexus · Jenkins

El hadaf mte3 ktib heda:
Nfehmou Kubernetes b *3a9l* moch b *7fadh*.
Chaque concept: *chnoua*, *3leh*, *kifeh*, *fel projet mte3na*.

Edition 2026 — v1.0

Contents

Mo9addima **9**

I Kubernetes Fundamentals — El Asesiyet **11**

1 Introduction: Chnoua Kubernetes w 3leh? **13**

1.1 El mochkla: 3leh 5lewna Kubernetes? . . . 13
1.2 Chnoua Kubernetes 9ala dénition . . . 13
1.2.1 Kubernetes = System Operating lel Data Center . . . 13
1.3 Container Orchestration: El Mochkla w El 7al . . . 14
1.3.1 9bal Kubernetes: Era el 3a7di . . . 14
1.3.2 Solutions historiques: Docker Swarm vs Mesos vs Kubernetes . . . 14
1.3.3 3leh Kubernetes rbe7? . . . 14
1.4 Waqtech nest3mlou Kubernetes w waqtech la? . . . 14
1.4.1 Use cases naj7in (nest3mlou K8s) . . . 15
1.4.2 Use cases khayyeb (MA nest3mlouch K8s) . . . 15
1.5 Roadmap mte3 el ktib heda . . . 15

2 Core Concepts — El Mafehim El Asesiya **17**

2.1 Pod — El Atom mte3 Kubernetes . . . 17
2.1.1 Kifeh y5addem Pod? . . . 17
2.1.2 Pod Lifecycle . . . 18
2.2 Node — El Machine (VM wla Physique) . . . 18
2.2.1 Master vs Worker . . . 19
2.3 Cluster — Majmou3a mta3 Nodes . . . 19
2.4 Namespace — T9sim Logique . . . 19
2.4.1 Namespaces bédéfaut . . . 20
2.5 Label & Selector — Kifeh Kubernetes Ylawej . . . 20
2.6 Deployment — El Manager mte3 Pods . . . 21
2.6.1 Example: nginx Deployment . . . 21
2.7 ReplicaSet — El Garde mte3 Replicas . . . 22
2.8 Service — Networking Stable . . . 22
2.8.1 4 Types mte3 Services . . . 22
2.9 Ingress — HTTP Routing . . . 23
2.10 ConfigMap & Secret — Configuration . . . 24

2.11 Volume & PersistentVolume — Storage . . . 25
2.11.1 Volume Types . . . 25
2.12 DaemonSet — Pod 3la Kol Node . . . 26
2.13 StatefulSet — lel Databases . . . 27
2.14 Job & CronJob — Tasks One-shot . . . 27

3 Architecture — El Mo5 w El 3odhlat 29
3.1 Control Plane — El Mo5 . . . 29
3.1.1 Architecture Overview . . . 29
3.1.2 API Server — El Báb El Wa7id . . . 29
3.1.3 etcd — El Memory mte3 El Cluster . . . 30
3.1.4 Scheduler — Chkoun Ya5tar El Node . . . 30
3.1.5 Controller Manager — El 7ares . . . 31
3.1.6 Cloud Controller Manager . . . 31
3.2 Worker Node Components — El 3odhlat . . . 31
3.2.1 kubelet — El Agent . . . 31
3.2.2 kube-proxy — Network Magic . . . 32
3.2.3 Container Runtime — containerd (moch Docker!) . . . 32
3.3 Add-ons — El Plugins . . . 33
3.3.1 CNI — Container Network Interface . . . 33
3.3.2 CoreDNS — Service Discovery . . . 33
3.3.3 Metrics Server — Ressources . . . 33

4 Networking — El Chabka 35
4.1 Network Model mte3 Kubernetes . . . 35
4.1.1 4 régles asesiyet . . . 35
4.1.2 Flat Network Model . . . 35
4.2 IP Ranges fel Projet Mte3na . . . 35
4.3 CNI Deep Dive: Flannel . . . 36
4.3.1 VXLAN Encapsulation . . . 36
4.3.2 Alternatives lel Flannel . . . 36
4.4 Service Discovery — DNS . . . 36
4.5 Network Policies — Firewall lel Pods . . . 37

5 Storage — El Stockage 39
5.1 El mochkla: Containers Ephemeral . . . 39
5.2 Volume Types Par Défault . . . 39
5.2.1 emptyDir — Temporaire . . . 39
5.2.2 hostPath — Node Filesystem . . . 39
5.2.3 NFS — Network Shared Storage . . . 40
5.3 PersistentVolume & PersistentVolumeClaim . . . 40
5.3.1 El Pattern . . . 40
5.3.2 Access Modes . . . 40
5.3.3 StorageClass — Dynamic Provisioning . . . 40

6 Security — El Sécurité **41**
6.1 Authentication & Authorization . . . 41
6.1.1 Chkoun tá5atab m3a API? . . . 41
6.1.2 ServiceAccount — Identity lel Pods . . . 41
6.1.3 RBAC — Role-Based Access Control . . . 41
6.2 Secrets Management . . . 42
6.2.1 Solutions pour Secrets . . . 42
6.3 Pod Security Standards . . . 42
6.4 Network Security . . . 43

7 Workload Management **45**
7.1 Deployment Strategies . . . 45
7.1.1 Rolling Update (default) . . . 45
7.1.2 Recreate . . . 45
7.1.3 Blue/Green . . . 45
7.1.4 Canary . . . 45
7.2 Scaling . . . 45
7.2.1 Manual Scaling . . . 45
7.2.2 HPA — Horizontal Pod Autoscaler . . . 46
7.2.3 VPA — Vertical Pod Autoscaler . . . 46
7.2.4 Cluster Autoscaler . . . 46
7.3 Resource Management . . . 46
7.3.1 QoS Classes . . . 46

8 Observability — Monitoring **47**
8.1 Logging . . . 47
8.1.1 kubectl logs — Basic . . . 47
8.1.2 Centralized Logging — EFK / Loki . . . 47
8.2 Metrics — Prometheus . . . 47
8.3 Tracing . . . 48

II Tools & Project Integration **49**

9 Infrastructure as Code **51**
9.1 Vagrant — VM Automation . . . 51
9.1.1 Vagrantfile Structure . . . 51
9.1.2 Commands Essentielles . . . 51
9.2 Ansible — Configuration Management . . . 52
9.2.1 Concepts Clés . . . 52
9.2.2 Fel Projet Mte3na . . . 52
9.2.3 Idempotency Example . . . 53
9.3 kubeadm — Cluster Bootstrap . . . 53
9.3.1 Flow mte3 kubeadm fel Projet . . . 53

10 CI/CD Stack — Gitea + Nexus + Jenkins **55**
10.1 Vue Globale: Complete CI/CD Flow . . . 55
10.2 Gitea — Self-hosted Git . . . 55
10.3 Nexus — Artifact Repository . . . 55
10.3.1 Docker Registry Setup . . . 56
10.4 Jenkins — CI/CD . . . 56
10.4.1 Plugins Nécessaires . . . 56
10.4.2 Pipeline Example . . . 57

11 Storage Solution — NFS **59**
11.1 NFS Server fel Projet Mte3na . . . 59
11.2 Exemple: MySQL m3a NFS . . . 59
11.3 Alternatives lel NFS . . . 60

III Practical Skills **63**

12 kubectl Mastery **65**
12.1 Essential Commands . . . 65
12.2 Flags Critiques . . . 65
12.3 Power Commands . . . 66
12.4 kubectl Aliases (Productivity) . . . 67
12.5 Debugging Techniques . . . 67
12.5.1 Pod Stuck en Pending . . . 67
12.5.2 CrashLoopBackOff . . . 67
12.5.3 ImagePullBackOff . . . 67

13 YAML Manifests Mastery **69**
13.1 Structure Standard . . . 69
13.2 Best Practices . . . 69
13.3 Probes — Health Checks . . . 69

14 Troubleshooting Guide **71**
14.1 Node Issues . . . 71
14.1.1 Node NotReady . . . 71
14.1.2 Disk Pressure . . . 71
14.2 Pod Issues . . . 72
14.2.1 Decision Tree . . . 72
14.3 Network Issues . . . 72
14.3.1 DNS Ma Y5adchi . . . 72
14.3.2 Service Ma Yo7ouzch Traffic . . . 72
14.3.3 Cross-node Communication (CNI) . . . 72
14.4 Cluster-Wide Debugging . . . 73

15 Production Readiness **75**
15.1 High Availability . . . 75
15.1.1 Control Plane HA . . . 75
15.1.2 Application HA . . . 75
15.2 Backup & Disaster Recovery . . . 75
15.2.1 etcd Snapshots . . . 75
15.2.2 Velero — Cluster Backup . . . 76
15.2.3 GitOps — Declarative Recovery . . . 76
15.3 Security Hardening . . . 76
15.4 Monitoring & Alerting . . . 76

IV Next Steps **79**

16 Learning Path **81**
16.1 Roadmap lel 6 Mois . . . 81
16.2 Certifications . . . 81

17 Hands-on Exercises **83**
17.1 Level 1: Basics . . . 83
17.2 Level 2: Intermediate . . . 83
17.3 Level 3: Advanced . . . 83
17.4 Level 4: Expert . . . 84

18 Resources **85**
18.1 Official . . . 85
18.2 Books . . . 85
18.3 Free Learning . . . 85
18.4 Communities . . . 85
18.5 Tools tḱsebha . . . 86

Conclusion **87**

Mo9addima

Ya sa7bi, ken enti ta9ra ktib heda, ya5i 3andek fekra 3la Docker wla 3andek cluster 5edem bel Vagrant ki mte3na, wla ghir 7abb tef7em `Kubernetes`. Kolha behi.

3leh el ktib heda ma5telef?

Barcha documentation 3la Kubernetes mawjouda online, ama fi ktib heda:

- **Kol concept** n7akiw 3leh b *4 questions*: *chnoua*, *3leh mawjoud*, *kifeh y5addem*, *fel projet mte3na kifeh*.
- **El commands** kol wa7ed menhom m3a explication chnoua y3ni.
- **El projet mte3na** (4 VMs: master + 2 workers + services) hiya el *terrain* bech nfehmou kol 7aja 3la 7ssab.
- **Bel darija**: kelmet arabiya b transliteration , =3) , =7 , =9 , =8 ,(=5 kelmet français, w **el technical terms tob9a b English** kima lezem.

Kifeh ta9ra el ktib heda

- Lou ken enti **beginner**: 9ra Part 1 b tertib, ma t5alliwch chapter.
- Lou ken enti **3andek ex3érience m3a Docker**: tnajem tebda men Chapter 2 (Core Concepts) w taqfez 3al Architecture.
- Lou ken enti **7abb tawel tefhem el projet mte3na**: Part 2 kolo 3la el tools (Vagrant, Ansible, Gitea, Nexus, Jenkins).
- Lou ken enti **bech ta3mel demo**: rou7 Part 3 Chapter 14 (kubectl mastery) w Chapter 16 (Troubleshooting).

Note

Rule #1 fel Kubernetes: *Ma te7fedhch, efhem!* `kubectl` 3andha 100+ commands. 7ad ma ya7fedhom kol. El mohim tefhem el model (Pod, Service, Deployment...) w ba3d el documentation `kubectl --help` tkafi.

Part I

Kubernetes Fundamentals — El Asesiyet

Chapter 1

Introduction: Chnoua Kubernetes w 3leh?

1.1 El mochkla: 3leh 5lewna Kubernetes?

Tsawar m3aya hedhi el situation: 3andek *micro-service* mektoub bel Node.js, w 7abbit t5addmou 3al production.

1. Bedit ya5dem m3a `Docker`: `docker run my-app`. Behi.
2. El service wlet 3andou traffic barcha. Lezmek 3 copies (*n3amlou replicas*).
3. `Docker Swarm`? Ken mawjoud ama limité. W lou ken VM tmout? El pods ykhosrou.
4. Kifeh n3amlou **rolling update** sans downtime?
5. Kifeh nsité3mlou **NFS storage** w yochar3ou 3al replicas?
6. Kifeh n7akou m3a microservices o5rin b **service discovery**?

Kol ma zedt fel scale, kol ma el mochakel twelli akther. **Kubernetes** je jébed el solution: platform té5adem kol hedha *awtomatik*.

1.2 Chnoua Kubernetes 9ala dénition

[

Kubernetes (K8s)] **Kubernetes** (isakha 5amm: *K8s* — 8 7arf bin K w S) hiya *container orchestration platform* open-source, 5arjet men Google (Borg project), mawjouda deldhi t7at el `CNCF` (Cloud Native Computing Foundation).

El job mte3ha: ta5dem containers fi cluster mta3 machines, *tdir 3lihom*, tsay7ihom ki ymoutou, ta3mel scale, networking, storage, kol 7aja.

1.2.1 Kubernetes = System Operating lel Data Center

Tsawar Kubernetes kima Linux ama lel cluster:

- Linux ydir fi **processes** (fork, schedule, kill) 3la machine we7da.
- Kubernetes ydir fi **pods** (create, schedule, restart) 3la cluster mta3 machines.

1.3 Container Orchestration: El Mochkla w El 7al

1.3.1 9bal Kubernetes: Era el 3a7di

9bal Kubernetes (w 9bal Docker 7ata), el applications kaínou t5edem 3la *VM monolithique*. Update = downtime. Scaling = tchéri machine jdida.

Ba3d je `Docker` (2013) w rj3et el containers mouch 7aja teoretique. Ama mochkla jdida:

- Kifeh n7akou bin 50 container?
- Win ne7atouhom (kol container 3la anhi machine)?
- Ki ymout container, chkoun y3awdou?
- Kifeh n3amlou upgrade sans downtime?

1.3.2 Solutions historiques: Docker Swarm vs Mesos vs Kubernetes

Solution	Mizatha	Mochakelha
`Docker Swarm`	Béssat, integrated m3a Docker	Features limités, community *matet*
`Apache Mesos`	Scale kbir (Twitter, Airbnb)	Complex, learning curve *qatila*
`Nomad` (HashiCorp)	Béssat, binary we7da	Ecosystem petit compared to K8s
`Kubernetes`	Standard industry, ecosystem enorme	Complex lel beginners

1.3.3 3leh Kubernetes rbe7?

1. **Backed by Google**: 15+ ans mte3 experience m3a Borg.
2. **Open Source + CNCF**: Ma thkomchi charika we7da, governance ouverte.
3. **Extensible**: CRDs, Operators, Webhooks — tnajem tzid 3lih chnoua ma 7abbit.
4. **Ecosystem**: Helm, Prometheus, Istio, ArgoCD — kolhom Kubernetes-native.
5. **Cloud Support**: Kol cloud kbir 3andou managed K8s (EKS, GKE, AKS).

1.4 Waqtech nest3mlou Kubernetes w waqtech la?

Tip Pratique

Kubernetes mouch toujours el solution. T5edem mégha applications moderne scalable. Ama lou ken 3andek *site statique* wla *1 container*, Docker wa7dou yekfi.

1.4.1 Use cases naj7in (nest3mlou K8s)

- **Microservices**: Apps fihom 10+ services b languages mo5telfin.
- **High Availability**: Lezem el service mayet9afelch abadan.
- **Auto-scaling**: Traffic yetghayyar (e.g., e-commerce fel soldes).
- **Multi-environment**: Dev, Staging, Prod 3la nafs el infrastructure.
- **CI/CD moderne**: Deploy 3la demande, rollback féssa3.

1.4.2 Use cases khayyeb (MA nest3mlouch K8s)

- **Monolithic app b 1 service**: Docker Compose yekfi.
- **Team ma ya3rafch DevOps**: K8s complex, ta7 ti wa9t 3al learning curve.
- **Budget petit**: Managed K8s (EKS/GKE) y5alef akther mel simple VM.
- **Stateful apps bessif**: Databases fel K8s mumkinet ama mouch *best practice* toujours.

1.5 Roadmap mte3 el ktib heda

Fi **Part 1**, bech n7akou 3la el fundamentals: Pods, Nodes, Services, Deployments — el bricks asesiyet. Ba3d n'a9r'ou 3al Architecture (API Server, etcd, Scheduler...). Networking w Storage w Security 7assin lezmet.

Fi **Part 2**, bech nchouf el tools: Vagrant, Ansible, containerd, Flannel, Gitea, Nexus, Jenkins — kolhom fel projet mte3na.

Fi **Part 3**, *5din 3al pratique*: kubectl mastery, YAML manifests, troubleshooting, production readiness.

Fi **Part 4**, kifeh tkammel learning — certifications, resources, next steps.

Chapter 2

Core Concepts — El Mafehim El Asesiya

Hedha chapter el akher ahammiya. Lou ken ma tefhemch el concepts heni, kol ba9i el ktib ma yod5olch l'dma8ek. 5din wa9tek.

2.1 Pod — El Atom mte3 Kubernetes

[

Pod] **Chnoua:** A9sar 7aja fel Kubernetes. Pod = *wa7ed wla akther* container, sharing network (IP we7da) w storage (volumes).

3leh: Kubernetes ma y5addemch containers directly, y5addem Pods. *3leh hakka?* 5atraha applications certaines 7ajetha containers *toumet* y5admou m3a ba3dhom (sidecar pattern).

Fel projet mte3na: CoreDNS pod, Flannel pod, kube-proxy pod, etcd pod — kolhom Pods.

2.1.1 Kifeh y5addem Pod?

Listing 2.1: Pod béssat

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-nginx
  labels:
    app: nginx
spec:
  containers:
  - name: nginx
    image: nginx:alpine
    ports:
    - containerPort: 80
```

Commands 3ladaha:

```bash
# Ta3mel Pod men YAML
kubectl apply -f my-pod.yaml

# Tchouf el pods mawjoudin
```

```
kubectl get pods

# Details mte3 pod
kubectl describe pod my-nginx

# Logs mte3 container fel pod
kubectl logs my-nginx

# D5el 9odhd a pod (kima SSH)
kubectl exec -it my-nginx -- /bin/sh

# Ame7ha
kubectl delete pod my-nginx
```

2.1.2 Pod Lifecycle

Pod yedh5al fi phases:

1. `Pending`: Ma t5attetch 3la node ba3d (image ma tétle3letch, wla ma fama node).
2. `Running`: Container y5addem.
3. `Succeeded`: Container kemal, exit 0 (Jobs).
4. `Failed`: Container tawa exit != 0.
5. `Unknown`: Kubernetes ma ta3arfch chnoú y saret (node problem).

Attention!

Ma ta3melch Pods directly! 7ta lou ken tnajjem. Pod ki ymout, mánouch y3awed. Est3mel `Deployment` bech Kubernetes y3awdńom awtomatik.

2.2 Node — El Machine (VM wla Physique)

[

Node] **Chnoua:** Machine (VM wla bare-metal) tidor fiha Pods. Node y7ouz `kubelet`, `kube-proxy`, w container runtime.

3leh: El pods lezemhom maken y5admou fih.

Fel projet mte3na: 3 nodes:

- `k8s-master` (192.168.56.10): control plane
- `k8s-worker1` (192.168.56.11): worker
- `k8s-worker2` (192.168.56.12): worker

2.2.1 Master vs Worker

Node Type	Components	Y5addem pods user?
Master (Control Plane)	API Server, etcd, Scheduler, Controller Manager	*Par défaut LA* (taint NoSchedule)
Worker	kubelet, kube-proxy, containerd	*AY* (3la daham n7attou apps)

```
# Tchouf nodes mawjoudin
kubectl get nodes

# Details kemla
kubectl get nodes -o wide

# Ken node NotReady, 3layh diagnostic
kubectl describe node k8s-worker1

# Cordonh (ma ya7atch 3lih pods jdod)
kubectl cordon k8s-worker1

# Uncordon (ya3aud ya5dem)
kubectl uncordon k8s-worker1

# Drain (ya7aued pods mel node --- mohim 9bal maintenance)
kubectl drain k8s-worker1 --ignore-daemonsets
```

2.3 Cluster — Majmou3a mta3 Nodes

[

Cluster] **Chnoua:** Majmou3a mta3 Nodes t5addem m3a ba3dhom ka unit we7da, tajm3hom el control plane.

3leh: High availability + scalability. Lou ken node ymout, el pods y3awdou 3la node okhra.

Fel projet mte3na: Cluster b 3 nodes (1 master + 2 workers). *Single master* (ma7na dev environment, ma3andnach HA).

2.4 Namespace — T9sim Logique

[

Namespace] **Chnoua:** Namespace = *folder virtuel* fel cluster. Ya3rez *ysalik* resources (pods, services, secrets...) lef 3a9la wa7id.

3leh: Bech t9assam el cluster bin teams wla environments (dev, staging, prod) sans cluster

jdid.

Fel projet mte3na: `kube-system` (system pods), `default` (nos apps), `kube-flannel` (CNI).

2.4.1 Namespaces bédéfaut

- `default`: Resources par défaut.
- `kube-system`: System pods (coredns, kube-proxy...). *Ma t5assarch fih.*
- `kube-public`: Public, readable mel kol el users.
- `kube-node-lease`: Node heartbeats.

```
# List namespaces
kubectl get namespaces

# Resources fi namespace
kubectl get pods -n kube-system

# Kol el namespaces
kubectl get pods -A

# Ta3mel namespace jdid
kubectl create namespace my-app

# Default namespace lel session
kubectl config set-context --current --namespace=my-app
```

2.5 Label & Selector — Kifeh Kubernetes Ylawej

[

Label] **Chnoua:** Key-value pair (e.g., `app=nginx`, `env=prod`) ta3bet 3la resources.

3leh: Kubernetes ma ye7kemch b names. Ye7kem b *groupes*. Labels hiya el way to make groups.

Kifeh y5addem: Service, Deployment, w controllers o5rin ylawejhou 3la pods b **selector**.

```yaml
metadata:
  labels:
    app: web
    tier: frontend
    env: production
```

```
# Kol pods b label app=nginx
kubectl get pods -l app=nginx

# Multiple labels (AND)
kubectl get pods -l app=nginx,env=prod
```

```
# NOT
kubectl get pods -l 'env!=dev'

# Add label 3ad live
kubectl label pod my-nginx version=v1
```

2.6 Deployment — El Manager mte3 Pods

[

Deployment] **Chnoua:** Object y3arref *chnoua 5erez bech y5addem* (image, replicas, update strategy).

3leh: Pods directly = hasheb. Deployment = declárative — entík tgoul "7abb 3 copies mte3 nginx", Kubernetes y5addem bech tkoun *toujours* 3 copies.

Kifeh y5addem: Deployment y5arez `ReplicaSet`, ReplicaSet y5arez `Pods`. Chaine mta3 ownership.

2.6.1 Example: nginx Deployment

Listing 2.2: nginx Deployment b 3 replicas

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.25
        ports:
        - containerPort: 80
        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"
          limits:
            cpu: "500m"
            memory: "256Mi"
```

```
# Deploy
kubectl apply -f nginx-deployment.yaml

# Scale
kubectl scale deployment nginx-deployment --replicas=5

# Update image (rolling update automatique)
kubectl set image deployment/nginx-deployment nginx=nginx:1.26

# Rollout status
kubectl rollout status deployment/nginx-deployment

# History
kubectl rollout history deployment/nginx-deployment

# Rollback
kubectl rollout undo deployment/nginx-deployment

# Rollback l'n revision specifique
kubectl rollout undo deployment/nginx-deployment --to-revision=2
```

2.7 ReplicaSet — El Garde mte3 Replicas

[

ReplicaSet] **Chnoua:** Controller ydammen nómro exact mel pods y5admou toujours.

3leh: Ki ymout pod, ReplicaSet y3awed jdid awtomatik.

Fel projet mte3na: Ma ta3mlouhach directly. **Deployment** ya5rezhalak.

2.8 Service — Networking Stable

[

Service] **Chnoua:** Object ya3tik *IP permanente* + DNS name lel groupe mte3 Pods.

3leh: Pods IPs yetghayyrou (ki ymout w ya3aud ya5rez, IP jdida). Service ya3tik *stable endpoint*.

Kifeh: Service ylawej b label selector 3la pods. `kube-proxy` y3amel iptables/ipvs rules.

2.8.1 4 Types mte3 Services

Type	Scope	Use Case
`ClusterIP` (default)	Internal only	Microservice-to-microservice

NodePort	External (port 30000-32767 3la kol node)	Dev, testing, simple external
LoadBalancer	External (cloud LB)	Production 3al cloud
ExternalName	DNS CNAME	Alias lśervice 5arej el cluster

Listing 2.3: Service ClusterIP

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: ClusterIP
  selector:
    app: nginx      # Ylawej 3la pods b label app=nginx
  ports:
  - port: 80        # Port mte3 service
    targetPort: 80 # Port mte3 pod
    protocol: TCP
```

Listing 2.4: Service NodePort (external)

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
  - port: 80
    targetPort: 80
    nodePort: 30080    # M\'etiid entre 30000-32767
```

```
# Access: http://<any-node-ip>:30080
curl http://192.168.56.10:30080
curl http://192.168.56.11:30080  # Ay node y5addem!
```

2.9 Ingress — HTTP Routing

[

Ingress] **Chnoua:** Routing rules HTTP/HTTPS mel 5arej lel services.

3leh: NodePort w LoadBalancer service per service = cher. Ingress = *reverse proxy* (nginx, traefik) y7ouz rules lel kol el apps.

Lezem: Ingress Controller (nginx-ingress, traefik) méctab fel cluster.

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: my-ingress
spec:
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: api-service
            port:
              number: 80
      - path: /
        pathType: Prefix
        backend:
          service:
            name: frontend-service
            port:
              number: 80
```

2.10 ConfigMap & Secret — Configuration

[

ConfigMap] **Chnoua:** Key-value storage lel *config data mouch sensitive* (URLs, feature flags, ports...).

3leh: Ma tháttchi config hard-coded fel image. Separation of concerns.

[

Secret] **Chnoua:** Kima ConfigMap ama lel *data sensitive* (passwords, tokens, certs). Base64 encoded (**mouch encrypted by default!**).

3leh: Separation + moins de risque 7tta lou tchouf manifest fel git.

```yaml
# ConfigMap
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  database_url: "postgres://db:5432/mydb"
  log_level: "INFO"
---
```

```yaml
# Secret
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  password: cGFzc3dvcmQxMjM=  # base64: password123
```

Listing 2.5: Injection fel Pod

```yaml
spec:
  containers:
  - name: app
    image: myapp:v1
    env:
    - name: DATABASE_URL
      valueFrom:
        configMapKeyRef:
          name: app-config
          key: database_url
    - name: DB_PASSWORD
      valueFrom:
        secretKeyRef:
          name: db-secret
          key: password
```

Attention!

Secret b base64 mouch sécurity! `echo cGFzc3dvcmQ= | base64 -d` = `password`. Enable *encryption at rest* fel etcd, wla est3mel Vault / Sealed Secrets lel production.

2.11 Volume & PersistentVolume — Storage

[

Volume] **Chnoua:** Directory m3a data, mounted fel container.

3leh: Container filesystem *ephemeral* — ki container ymout, data tédhra3. Volume étdáom.

2.11.1 Volume Types

Type	Description	Lifetime
`emptyDir`	Directory empty created ki pod yabda	Pod lifetime
`hostPath`	Directory mel node filesystem	Node lifetime
`configMap`	Mount ConfigMap as files	—
`secret`	Mount Secret as files	—

`nfs`	NFS share (*fel projet mte3na!*)	Permanent
`persistentVolumeClaim`	Claim 3la PV dynamique	Permanent

[

PersistentVolume (PV) + PersistentVolumeClaim (PVC)] **PV:** Storage resource fel cluster (like a "disk"). Admin y5al9ou.

PVC: Request mel user: "7abb 5 GB storage ReadWriteOnce". Kubernetes ylawej 3la PV métlaim.

Fel projet mte3na: NFS server 3al services VM. Tnajem ta3mel PV ypointer 3la `192.168.56.20:/srv/nfs/m`

Listing 2.6: PV + PVC m3a NFS

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nfs-pv
spec:
  capacity:
    storage: 5Gi
  accessModes:
    - ReadWriteMany
  nfs:
    server: 192.168.56.20
    path: /srv/nfs/mysql-data
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nfs-pvc
spec:
  accessModes:
    - ReadWriteMany
  resources:
    requests:
      storage: 5Gi
```

2.12 DaemonSet — Pod 3la Kol Node

[

DaemonSet] **Chnoua:** Controller ydaman pod we7ed y5addem 3la kol node fel cluster.

3leh: Services nódhbethom fi kol machine: logging agent, monitoring agent, CNI plugin...

Fel projet mte3na:

- `kube-flannel-ds`: Flannel CNI 3la kol node

- kube-proxy: 3la kol node

```
# Chouf les DaemonSets
kubectl get daemonsets -A
```

2.13 StatefulSet — lel Databases

[

StatefulSet] **Chnoua:** Kima Deployment ama lel applications *stateful* (7aajithom identity fixe, storage fixe, order fixe).

3leh: Databases: MySQL, PostgreSQL, MongoDB, Cassandra, Kafka — ma tnajjemch t3amelhom Deployment.

Features:

- Pods 3andhom names stables (`mysql-0`, `mysql-1`, `mysql-2`).
- Storage persistant per pod.
- Order mte3 startup/shutdown.

2.14 Job & CronJob — Tasks One-shot

[

Job] **Chnoua:** Run pod wa7ed wla akther l'n job ikmel (e.g., database migration, backup).

3leh: Deployment y5addem toujours. Job y5aden *marra*, ba3d y9aff.

[

CronJob] **Chnoua:** Job y5adem 3la schedule (cron syntax).

3leh: Tasks dawriya: backup kol yóum, cleanup kol smána...

```yaml
apiVersion: batch/v1
kind: CronJob
metadata:
  name: backup-job
spec:
  schedule: "0 2 * * *"    # Kol y\'oum el 2h
  jobTemplate:
    spec:
      template:
        spec:
          containers:
          - name: backup
            image: backup-tool:v1
            command: ["/backup.sh"]
```

```
            restartPolicy: OnFailure
```

Chapter 3

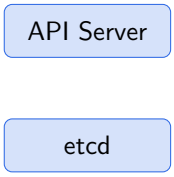
Architecture — El Mo5 w El 3odhlat

Fi chapter heda bech nefhmou *chnoua mawjoud* guédh chaque node, w kifeh el cluster y5addem mel dekhel.

3.1 Control Plane — El Mo5

El Control Plane = el components y5admou 3la master node. Howa el **brain** mte3 el cluster.

3.1.1 Architecture Overview



- `kube-scheduler`, `kube-controller-manager`: loopback.
- *Chaque pod* ken 7abb y7ki m3a K8s API: ServiceAccount token.

```
# Tchouf URL mte3 API server
kubectl cluster-info

# Raw API call
kubectl get --raw /api/v1/namespaces/default/pods

# curl direct (b token)
TOKEN=$(kubectl -n default create token default)
curl -k -H "Authorization: Bearer $TOKEN" \
  https://192.168.56.10:6443/api/v1/pods
```

3.1.3 etcd — El Memory mte3 El Cluster

[

etcd] **Chnoua:** Distributed key-value store, bésd consistent (Raft consensus). Tous les *state* mte3 el cluster fih.

3leh: Kubernetes stateless — ki ymout api-server, tá5dach. Ama ki ymout etcd, *kollou khsir*.

Chi yétsajel fih: kol object (Pods, Services, ConfigMaps, Secrets, Nodes...) — every single YAML manifest.

Attention!

etcd backup = *a9sar priority* fi production! Lou ken etcd cluster y5sar, ma tnajemch t3awed el cluster (unless 3andek backup). Schedule snapshots kol yóum.

```
# Snapshot mte3 etcd (3al master)
sudo ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key \
  snapshot save /backup/etcd-$(date +%F).db
```

3.1.4 Scheduler — Chkoun Ya5tar El Node

[

kube-scheduler] **Chnoua:** Ki pod jadid mawjoud (pending, ma3andouch node), scheduler yá5tar *anhi node* el monaseb.

Algorithm b simplification:

1. **Filtering**: Anhi nodes mumkinin? (CPU kéfi, RAM kéfi, taints compatible, nodeSelector)

2. **Scoring**: Anhi a7san? (node a9al loaded, affinity...)

Factors fel scheduling:

- **Resource requests** (CPU, memory): Node lezem yekkafi.
- **Node Selector**: `disk: ssd` — yet5ayyr ghir nodes 3andhom label heda.
- **Affinity / Anti-affinity**: *attire* wla *ab3eid* 3la pods.
- **Taints / Tolerations**: Node yǵouĺ "nombre limité", pod lezmou tolération.

3.1.5 Controller Manager — El 7ares

[

kube-controller-manager] **Chnoua:** Process y7ouz kol el controllers. Controller = loop yǵarń *desired state* vs *actual state*, w yśyil differences.

Types mte3 controllers:

- **Deployment Controller**: Ydamen replicas voulus.
- **ReplicaSet Controller**: Ydamen pods comptés.
- **Node Controller**: Ychóuf nodes, ymarkehom NotReady ken ma yrespondouch.
- **Job Controller**: Y5arez pods lel jobs.
- **Service Account Controller**: Y5arez default service accounts.
- ...w 30+ o5rin.

3.1.6 Cloud Controller Manager

Ken fel cloud (AWS, GCP, Azure), hedha el component y5addem m3a provider: provision Load-Balancers, attach volumes, manage nodes. Fel projet mte3na (Vagrant): *ma mawjoudch*.

3.2 Worker Node Components — El 3odhlat

3.2.1 kubelet — El Agent

[

kubelet] **Chnoua:** Agent y5addem 3la kol node. Y7ki m3a API server, ya5ou instructions, w y5adem pods via *container runtime*.

Job mte3ou:

1. Register el node m3a API server.
2. Watch pods métlas lu node heda.
3. Start/stop containers via CRI.

4. Report status (health checks, resource usage) lÁPI.

Fel projet mte3na: Config fi `/etc/kubernetes/kubelet.conf` + `/etc/default/kubelet`.

```
# Logs mte3 kubelet (linux systemd)
sudo journalctl -u kubelet -f

# Status
sudo systemctl status kubelet

# Restart (ken mochkla)
sudo systemctl restart kubelet
```

3.2.2 kube-proxy — Network Magic

[

kube-proxy] **Chnoua:** DaemonSet y5addem 3la kol node. Yńódhem networking rules (iptables wla ipvs) bech Services yemchiwu.

Kifeh y5addem: Ki ta3mel Service, kube-proxy y5atet iptables rules:

- Traffic lĆlusterIP `10.96.0.5` → forward lṕod IP `10.244.1.23:80`.
- Load balancing (round-robin) bin el pods.

Attention!

Fel projet mte3na *sar mochkla kbira*: kubeadm y5leq kube-proxy kubeconfig m3a IP mel *default-route interface* (NAT: 10.0.2.15), mouch `--apiserver-advertise-address` (192.168.56.10). Resultat: kube-proxy ma ynajjemch y7ki m3a API, Flannel yćracshé, workers NotReady. Fix: patch el ConfigMap. *Codified fel role master/tasks/main.yml.*

3.2.3 Container Runtime — containerd (moch Docker!)

[

containerd] **Chnoua:** Container runtime — software y5addem containers directly.

3leh moch Docker:

- Docker 3andou CLI + daemon + build — kbir.
- Kubernetes 7ajetou b GH runtime. Docker kan yést5dem `dockershim` (shim) bin kubelet w Docker.
- Depuis Kubernetes **1.24**, `dockershim` *enleve*. Containerd (wla CRI-O) asma mou baŕáha.
- Docker images = containerd images (OCI format), *compatibles*.

Fel projet mte3na:

- `SystemdCgroup = true` fel `/etc/containerd/config.toml`.
- `sandbox_image = "registry.k8s.io/pause:3.9"`.

[

CRI — Container Runtime Interface] **Chnoua:** API standard bin kubelet w container runtime.

3leh: Kubernetes ma 7abbchi yátblej m3a runtime we7ed. CRI = plugin interface. Ay runtime mo7taram CRI (containerd, CRI-O) ymkiní y5addem m3a K8s.

```
# Crictl --- CLI like docker ama lel CRI
sudo crictl ps          # Containers working
sudo crictl images      # Images
sudo crictl logs <id>   # Logs
```

3.3 Add-ons — El Plugins

3.3.1 CNI — Container Network Interface

[

CNI] **Chnoua:** Standard + plugins bech pods y7aki bénaat hom + m3a 5arej.

3leh: Kubernetes ma yńach networking implementation. T5ayer CNI plugin 3la 7sab needs.

Fel projet mte3na: `Flannel` (VXLAN overlay, béssat). Alternatives: Calico (BGP + policies), Cilium (eBPF, advanced).

3.3.2 CoreDNS — Service Discovery

[

CoreDNS] **Chnoua:** DNS server y5addem fi pods fel cluster.

3leh: Pods y3arfou y7akou m3a services b names: `my-service.default.svc.cluster.local` au lieu de IP.

Fel projet mte3na: 2 pods `coredns-*` fi kube-system namespace.

3.3.3 Metrics Server — Ressources

[

Metrics Server] **Chnoua:** Collect CPU/memory usage pour pods w nodes.

3leh: `kubectl top` ma t5addemch bidoun. Lezmou lel HPA (autoscaling).

Fel projet mte3na: *Ma méctabch.* Ken 7abbit test5dem `kubectl top nodes`, lezem tínstaller metrics-server.

Chapter 4

Networking — El Chabka

Heda el chapter el akher critique. Networking fel K8s s3ib áwal marra, ama ki tefhmou, el kol yewdh7.

4.1 Network Model mte3 Kubernetes

4.1.1 4 régles asesiyet

Kubernetes yfarar chroo7 network b regles *net*:

1. **Pod-to-Pod**: Kol pod 3andou IP unique. Pods y7akou directly, *sans NAT*, mel ay node.
2. **Pod-to-Service**: Via ClusterIP (virtual). kube-proxy ydir DNAT.
3. **External-to-Service**: Via NodePort, LoadBalancer, Ingress.
4. **Network Policies**: Firewall rules bin pods (defense-in-depth).

4.1.2 Flat Network Model

Tip Pratique

El différence el kbira vs Docker classique: Docker ta3mel NAT + port mapping. K8s LA. Kol pod 3andou *IP vraie* fel network 3am mte3 el cluster. T7aki m3a pod 3la node o5ra = kima t7aki m3a pod 3la nafs node.

4.2 IP Ranges fel Projet Mte3na

Network	CIDR	Usage
VM Network (host-only)	`192.168.56.0/24`	VMs inter-communication
Pod Network (Flannel)	`10.244.0.0/16`	Pod IPs (overlay)
Service Network	`10.96.0.0/12`	ClusterIP virtual IPs

Chnoua ya3ni hedha pratiquement:

- Node IPs: `192.168.56.10, .11, .12, .20`
- Pods 3la worker1 mumkin yekhdhou: `10.244.1.2, 10.244.1.3...`

- Pods 3la worker2: `10.244.2.2, 10.244.2.3...`
- Services: `10.96.0.1` (kubernetes API service), `10.96.0.10` (coredns)...

4.3 CNI Deep Dive: Flannel

4.3.1 VXLAN Encapsulation

Flannel y5addem b **VXLAN**: overlay network fouq UDP.

- Pod 3la worker1 (`10.244.1.5`) 7abb yŕasel pod 3la worker2 (`10.244.2.8`).
- Packet yet7att fi UDP packet jdid (encapsulation), yem'si men `192.168.56.11` l'`192.168.56.12`.
- Flannel 3al worker2 yđécapsulate, yŕassel lel pod cible.

Attention!

Mochkla fel dual-NIC (fel projet mte3na): VMs 3andhom 2 NICs: `enp0s3` (NAT) w `enp0s8` (host-only). Flannel default y5atar enp0s3, ma 7akchi m3a nodes o5rin. *Fix:* `--iface=enp0s8` fel Flannel manifest (fel role `cni`).

4.3.2 Alternatives lel Flannel

CNI	Strengths	Weaknesses
Flannel	Béssat, stable, 5atr learn	Ma fihoulsh Network Policies
Calico	Policies, BGP (mouch overlay), fast	Config plus complex
Cilium	eBPF, observability, policies L7	Kernel 4.9+, courbe d'apprentissage
Weave	P2P mesh, easy	Performance moins bonne

4.4 Service Discovery — DNS

Kol namespace 3andou DNS sous-dómaine. Format:

```
<service>.<namespace>.svc.cluster.local
```

Examples:

- `kubernetes.default.svc.cluster.local` → API server (10.96.0.1)
- `my-app.default.svc.cluster.local` → Service `my-app` fi default namespace
- Mel pod same namespace: `my-app` yekfi (short name)

```
# T5oud DNS mel pod
kubectl run -it --rm debug --image=busybox --restart=Never -- sh
/ # nslookup kubernetes
Server:    10.96.0.10
Address 1: 10.96.0.10 kube-dns.kube-system.svc.cluster.local

Name:      kubernetes
Address 1: 10.96.0.1 kubernetes.default.svc.cluster.local
```

4.5 Network Policies — Firewall lel Pods

[

NetworkPolicy] **Chnoua:** Rules bech t5tar *anhi pod ynajjem yaqra l'anhi pod.*

3leh: Defense-in-depth. Microsegmentation. Zero Trust.

Lezem: CNI méstajib (Calico, Cilium, Weave). *Flannel MA IMPLEMENTCH NetworkPolicies!*

Listing 4.1: Deny-all ingress par défault

```yaml
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata:
  name: default-deny-ingress
  namespace: default
spec:
  podSelector: {}
  policyTypes:
  - Ingress
```

Chapter 5

Storage — El Stockage

5.1 El mochkla: Containers Ephemeral

Container filesystem *ya5sar* ki container ymout. Lou ken 3andek database, logs, uploads... kollou yroh.

Solution: Volumes.

5.2 Volume Types Par Défault

5.2.1 emptyDir — Temporaire

```yaml
spec:
  containers:
  - name: app
    volumeMounts:
    - mountPath: /cache
      name: cache-vol
  volumes:
  - name: cache-vol
    emptyDir: {}
```

Lifetime: Pod lifetime. Ki pod ymout, data yénjráf.

5.2.2 hostPath — Node Filesystem

```yaml
volumes:
- name: data
  hostPath:
    path: /var/data
    type: DirectoryOrCreate
```

Attention!

hostPath *dangerous*! Pod yqra/yakteb fi filesystem mte3 el node — security risk. Data tied lńode — ki pod yetnaql, data ma yefotch m3ah. *Ma test5dmoch* sauf lel system pods.

5.2.3 NFS — Network Shared Storage

```yaml
volumes:
- name: mysql-data
  nfs:
    server: 192.168.56.20
    path: /srv/nfs/mysql-data
```

Tip Pratique

Fel projet mte3na: NFS server 3al services VM. Kol el nodes ynajmou mount `/srv/nfs/mysql-data`. *Perfect* lel `ReadWriteMany` use cases (multiple pods y9rawa w yketbou m3a ba3dhom).

5.3 PersistentVolume & PersistentVolumeClaim

5.3.1 El Pattern

1. **Admin**: Y5arez PVs (wla StorageClass pour dynamic provisioning).
2. **User**: Y5arez PVC (request: "7abb 5Gi, ReadWriteOnce").
3. **Kubernetes**: Yb́ind PVC m3a PV moul9em.
4. **Pod**: Yḿount el PVC.

5.3.2 Access Modes

- `ReadWriteOnce (RWO)`: Un seul node ynajm yẃrite.
- `ReadOnlyMany (ROX)`: Multiple nodes read-only.
- `ReadWriteMany (RWX)`: Multiple nodes read-write (NFS supports heda).

5.3.3 StorageClass — Dynamic Provisioning

[

StorageClass] **Chnoua:** Tódhbet *kifeh* yet5al9ou PVs awtomatik.

3leh: Admin ma y3amelch PVs manually. StorageClass + PVC = PV jdid awtomatik.

Examples: `aws-ebs-gp3`, `gce-pd-ssd`, `local-path` (dev), `nfs-provisioner`.

Chapter 6

Security — El Sécurité

6.1 Authentication & Authorization

6.1.1 Chkoun tá5atab m3a API?

3 questions lezem API Server yjawab 3lihom:

1. **Authentication**: *Chkoun enta?* (certificates, tokens, OIDC)
2. **Authorization**: *Chnoua 7a9ek ta3mel?* (RBAC, ABAC, Webhook)
3. **Admission Control**: *Request mte3ak valide?* (mutation + validation)

6.1.2 ServiceAccount — Identity lel Pods

[

ServiceAccount] **Chnoua:** Account *non-human* yést5dḿoulha les pods lýet7akou m3a API server.

3leh: Chaque pod yelzmou identity lou yeqra secrets wla y3amel actions fel cluster.

Par défault: Kol namespace fih `default` ServiceAccount. Pod y5odhha ken ma tódb̓tśh wa7ed o5ra.

6.1.3 RBAC — Role-Based Access Control

[

RBAC] **4 concepts:**

- `Role`: Permissions fi namespace specifique.
- `ClusterRole`: Permissions 3la kol el cluster.
- `RoleBinding`: Rabt Role m3a User/Group/SA fi namespace.
- `ClusterRoleBinding`: Rabt cluster-wide.

Listing 6.1: Read-only Role lel pods

```
apiVersion: rbac.authorization.k8s.io/v1
```

```yaml
kind: Role
metadata:
  name: pod-reader
  namespace: default
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: bind-pod-reader
  namespace: default
subjects:
- kind: ServiceAccount
  name: my-sa
  namespace: default
roleRef:
  kind: Role
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

6.2 Secrets Management

Attention!

Secrets fel K8s b base64 *mouch* encrypted par défault! *Lezem* ténable *encryption at rest* fel etcd, wla tést5dem solutions o5rin.

6.2.1 Solutions pour Secrets

- **etcd encryption at rest**: Minimum hygiene.
- **HashiCorp Vault**: Gold standard. Dynamic secrets, audit, rotation.
- **Sealed Secrets** (Bitnami): Encrypt secrets fel Git (GitOps-friendly).
- **External Secrets Operator**: Sync secrets men Vault/AWS SM/GCP SM.

6.3 Pod Security Standards

3 profiles officiels (remplacent PodSecurityPolicies dhréou):

Profile	Description
`Privileged`	Kol 7aja mumkin. *Ma tést5dmoch fel prod.*

`Baseline`	Restrictions minimales (no privileged containers, no host namespaces).
`Restricted`	Highly restricted (non-root, read-only fs, no capabilities).

6.4 Network Security

Déjá vu: NetworkPolicies (chapter 4). **Rules:**

- Default deny all.
- Explicit allow per namespace/service.
- Ingress + egress controls.

Chapter 7

Workload Management

7.1 Deployment Strategies

7.1.1 Rolling Update (default)

[

Rolling Update] Kubernetes yŕeplaces pods *tédríja* — ya5erz pod jdid, ya7aut a9odaim ba3d test.

Zero downtime ken configuré b readiness probes + maxUnavailable + maxSurge.

```yaml
spec:
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxUnavailable: 1    # 1 pod out 3al ma5oud
      maxSurge: 1          # 1 pod extra allowed during update
```

7.1.2 Recreate

Kill kol el pods, ba3d y5arez new. **Downtime mawjoud**. Use case: apps qui ne peuvent pas coexist v1 + v2.

7.1.3 Blue/Green

Deux environnements complets (blue = current, green = new). Switch traffic b service selector. *Ma tíntnach mech native*, tnajem ta3mlou m3a labels.

7.1.4 Canary

Deploy versionne jdída lńombre kańagh pods (e.g., 10%). Monitor. Ken behi, roll out 3al koll.

7.2 Scaling

7.2.1 Manual Scaling

```
kubectl scale deployment nginx --replicas=10
```

7.2.2 HPA — Horizontal Pod Autoscaler

[

HPA] **Chnoua:** Controller ya5rej pods awtomatik 3la 7saab CPU wla memory usage (wla custom metrics).

Lezem: Metrics Server méctab.

```
kubectl autoscale deployment nginx --min=2 --max=10 --cpu-percent=70
```

7.2.3 VPA — Vertical Pod Autoscaler

Modify requests/limits dún pod (mouch nombre, mais taille). Less common.

7.2.4 Cluster Autoscaler

Add/remove *nodes* 3al demande (cloud only).

7.3 Resource Management

[

Requests vs Limits] **Request:** A9al quantity guaranteed lel container. Scheduler yést5dmhou bech y9arar win y7at el pod.

Limit: Maximum quantity. Ken container yetjaouz, CPU ypghńich, memory → *OOMKilled*.

```yaml
resources:
  requests:
    cpu: "250m"       # 0.25 CPU core
    memory: "128Mi"
  limits:
    cpu: "500m"
    memory: "256Mi"
```

7.3.1 QoS Classes

Class	Condition	Eviction priority
`Guaranteed`	requests == limits (kol containers)	Last to be killed
`Burstable`	requests < limits	Middle
`BestEffort`	Ma 3andouch requests/limits	First to be killed

Chapter 8

Observability — Monitoring

8.1 Logging

8.1.1 kubectl logs — Basic

```
kubectl logs my-pod                  # Current container
kubectl logs my-pod --previous       # Previous container (crash)
kubectl logs my-pod -c sidecar       # Specific container
kubectl logs -f my-pod               # Follow (tail)
kubectl logs -l app=nginx            # All pods m3a label
```

8.1.2 Centralized Logging — EFK / Loki

- **EFK Stack**: Elasticsearch + Fluentd + Kibana. Classic, heavy.
- **Loki**: By Grafana Labs. Lightweight, indexes labels not content. Cheaper.
- **Vector**: Modern agent, observability pipelines.

8.2 Metrics — Prometheus

[

Prometheus] **Chnoua:** Time-series database, scrapes metrics mel targets (pods b `/metrics` endpoint).

3leh el standard: CNCF graduated, integrated everywhere, PromQL powerful.

Stack typique:

- **Prometheus**: Collect + store metrics.
- **Grafana**: Visualize (dashboards).
- **Alertmanager**: Send alerts (email, Slack, PagerDuty).
- **Node Exporter**: Node metrics (CPU, RAM, disk).
- **kube-state-metrics**: K8s objects metrics.

Tip Pratique

Easiest way: `kube-prometheus-stack` Helm chart. Install one command, kóssou gratis: Prometheus + Grafana + Alertmanager + dashboards + alerts par défault.

8.3 Tracing

Distributed tracing: *track requests 3abra microservices*. Tools:

- **Jaeger**: CNCF, compatible OpenTelemetry.
- **Zipkin**: Twitter's, older.
- **OpenTelemetry**: Standard moderne lel instrumentation.

Part II

Tools & Project Integration

Chapter 9

Infrastructure as Code

9.1 Vagrant — VM Automation

[

Vagrant] **Chnoua:** Tool mte3 HashiCorp láutomate provisioning mte3 VMs locales (VirtualBox, VMware, Hyper-V).

3leh: Sans Vagrant lezmek manually: ta3mel VM, install OS, config network, install tools. B Vagrant = `vagrant up` command we7da.

Fel projet mte3na: 4 VMs (1 master + 2 workers + services), kol wa7da b^{c}onfig network static.

9.1.1 Vagrantfile Structure

Listing 9.1: Vagrantfile example

```
Vagrant.configure("2") do |config|
  config.vm.box = "ubuntu/jammy64"

  config.vm.define "k8s-master" do |master|
    master.vm.hostname = "k8s-master"
    master.vm.network "private_network", ip: "192.168.56.10"
    master.vm.provider "virtualbox" do |vb|
      vb.memory = 4096
      vb.cpus = 2
    end
  end
  # ... workers, services
end
```

9.1.2 Commands Essentielles

```
vagrant up              # Start + provision
vagrant halt            # Stop (preserve state)
vagrant reload          # Restart
vagrant destroy -f      # Delete everything
vagrant ssh <vm-name>   # SSH into VM
```

```
vagrant status          # Show all VMs state
vagrant provision       # Re-run Ansible
```

9.2 Ansible — Configuration Management

[

Ansible] **Chnoua:** Configuration management + automation tool. YAML-based, agentless (works via SSH).

3leh moch Bash scripts:

- **Idempotency**: Run 100 times = same result.
- **Declarative**: Tgoul *chnoua 7abb*, mouch *kifeh*.
- **Readable**: YAML vs bash spaghetti.
- **Modules**: 3000+ built-in (apt, service, template, file, etc.).

9.2.1 Concepts Clés

Concept	Description
Inventory	List mte3 hosts + groups (`inventory.ini`).
Playbook	YAML file, ysequence plays (`playbook.yml`).
Play	Majmou3a mte3 tasks lún groupe d'hosts.
Role	Reusable collection mte3 tasks + handlers + templates.
Task	Action we7da (install package, copy file...).
Handler	Task ma ya5dmch ken *notified* (restart service).
Module	Function built-in (`apt`, `copy`, `shell`...).
Variable	Configurable value (`group_vars/all.yml`).
Template	Jinja2 file generated on-the-fly.

9.2.2 Fel Projet Mte3na

```
ansible/
 inventory.ini              # k8s-master, k8s-worker1, k8s-worker2, services
 group_vars/
    all.yml                # kube_version, master_ip, pod_network_cidr...
 roles/
    common/                # swap, kernel modules, sysctl, packages
    containerd/            # containerd install + SystemdCgroup
    kubernetes/            # kubeadm, kubelet, kubectl
    master/                # kubeadm init + kube-proxy fix
    workers/               # kubeadm join
    cni/                   # Flannel manifest
```

```
    nfs/, docker/, gitea/, nexus/, jenkins/
playbook.yml                # Main orchestration
```

9.2.3 Idempotency Example

Listing 9.2: Mel master role mte3na

```
- name: Check if Kubernetes is already initialized
  ansible.builtin.stat:
    path: /etc/kubernetes/admin.conf
  register: kubeadm_init_check

- name: Run kubeadm init
  ansible.builtin.command: >
    kubeadm init --apiserver-advertise-address={{ master_ip }}
    --pod-network-cidr={{ pod_network_cidr }}
  when: not kubeadm_init_check.stat.exists   # <-- Idempotency guard
```

9.3 kubeadm — Cluster Bootstrap

[

kubeadm] **Chnoua:** Official CLI tool béch tbootstrap Kubernetes cluster.

Alternatives:

- **kops**: AWS-focused, production-ready.
- **kubespray**: Ansible-based, multi-cloud.
- **Rancher**: Full platform m3a UI.
- **Managed**: EKS (AWS), GKE (Google), AKS (Azure).
- **minikube / kind**: Local dev only.

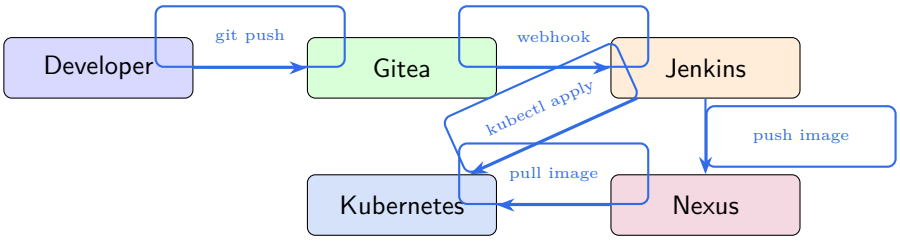
9.3.1 Flow mte3 kubeadm fel Projet

1. **Master**: `kubeadm init --apiserver-advertise-address=192.168.56.10 --pod-network-cidr=10.244.0.0/16`
2. **Master**: `kubeadm token create --print-join-command` → output command
3. **Workers**: Run that command (includes token + CA hash)
4. **Master**: Apply CNI (Flannel) manifest

Chapter 10

CI/CD Stack — Gitea + Nexus + Jenkins

10.1 Vue Globale: Complete CI/CD Flow



10.2 Gitea — Self-hosted Git

[

Gitea] **Chnoua:** Git server open-source, lightweight (Go). Alternative self-hosted lĠitHub/GitLab.

Fel projet mte3na: Port 3000, deployé b Docker Compose 3la services VM.

Features:

- Git hosting (repos, branches, PRs)
- Issues + Wiki
- Webhooks (trigger Jenkins)
- CI/CD (Gitea Actions)

URL: `http://192.168.56.20:3000`

10.3 Nexus — Artifact Repository

[

Nexus Repository Manager] **Chnoua:** Binary repository manager. Fi wáhid, ystókri 3áda types:

- Docker images (registry)
- Maven JARs

- npm packages
- PyPI
- Raw files

Fel projet mte3na:

- Port 8081: Nexus UI
- Port 8082: Docker registry endpoint

10.3.1 Docker Registry Setup

1. Login lŃexus UI (`http://192.168.56.20:8081`) m3a `admin` + password mel container.
2. Créer *Docker hosted repository* (`docker-hosted`).
3. Assign HTTP port 8082.
4. Sur Docker hosts: `/etc/docker/daemon.json`:

```
{
  "insecure-registries": ["192.168.56.20:8082"]
}
```

5. Push image:

```
docker tag myapp:v1 192.168.56.20:8082/myapp:v1
docker login 192.168.56.20:8082
docker push 192.168.56.20:8082/myapp:v1
```

10.4 Jenkins — CI/CD

[

Jenkins] **Chnoua:** Automation server. Pipelines (Groovy), plugins (2000+), community enorme.

Fel projet mte3na: Port 8080, initial password printed during provisioning.

Alternatives plus modernes:

- **GitLab CI**: Integrated m3a GitLab.
- **GitHub Actions**: Integrated m3a GitHub.
- **ArgoCD**: GitOps lel Kubernetes (declarative).
- **Tekton**: Kubernetes-native pipelines.

10.4.1 Plugins Nécessaires

- **Kubernetes plugin**: Run agents as pods.

- **Docker plugin**: Build + push images.
- **Git plugin**: Clone repos.
- **Credentials plugin**: Manage secrets.
- **Blue Ocean**: Modern UI.

10.4.2 Pipeline Example

Listing 10.1: Jenkinsfile complet

```
pipeline {
  agent any
  environment {
    REGISTRY = '192.168.56.20:8082'
    IMAGE    = 'myapp'
  }
  stages {
    stage('Checkout') {
      steps {
        git url: 'http://192.168.56.20:3000/user/myapp.git'
      }
    }
    stage('Build') {
      steps {
        sh "docker build -t ${REGISTRY}/${IMAGE}:${BUILD_NUMBER} ."
      }
    }
    stage('Push') {
      steps {
        withCredentials([usernamePassword(
          credentialsId: 'nexus-creds',
          usernameVariable: 'USER',
          passwordVariable: 'PASS')]) {
          sh "echo $PASS | docker login -u $USER --password-stdin ${REGISTRY}"
          sh "docker push ${REGISTRY}/${IMAGE}:${BUILD_NUMBER}"
        }
      }
    }
    stage('Deploy') {
      steps {
        sh """
          kubectl set image deployment/myapp \
            myapp=${REGISTRY}/${IMAGE}:${BUILD_NUMBER} \
            --record
        """
      }
    }
  }
}
```

Chapter 11

Storage Solution — NFS

11.1 NFS Server fel Projet Mte3na

[

NFS (Network File System)] **Chnoua:** Protocol yallaw mount filesystem distant comme s'il était local.

3leh fel K8s: `ReadWriteMany` — multiple pods 3la different nodes ynajmou yaqrawa w yketbou 3la nafs el data.

Fel projet mte3na:

- Services VM: `192.168.56.20`
- Export: `/srv/nfs/mysql-data`
- Allowed network: `192.168.56.0/24`

11.2 Exemple: MySQL m3a NFS

```yaml
# PV
apiVersion: v1
kind: PersistentVolume
metadata:
  name: mysql-pv
spec:
  capacity:
    storage: 10Gi
  accessModes: [ReadWriteMany]
  nfs:
    server: 192.168.56.20
    path: /srv/nfs/mysql-data
---
# PVC
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: mysql-pvc
```

```yaml
spec:
  accessModes: [ReadWriteMany]
  resources:
    requests:
      storage: 10Gi
---
# Deployment
apiVersion: apps/v1
kind: Deployment
metadata:
  name: mysql
spec:
  replicas: 1
  selector:
    matchLabels: {app: mysql}
  template:
    metadata:
      labels: {app: mysql}
    spec:
      containers:
      - name: mysql
        image: mysql:8.0
        env:
        - name: MYSQL_ROOT_PASSWORD
          valueFrom:
            secretKeyRef:
              name: mysql-secret
              key: root-password
        volumeMounts:
        - name: data
          mountPath: /var/lib/mysql
      volumes:
      - name: data
        persistentVolumeClaim:
          claimName: mysql-pvc
```

11.3 Alternatives lel NFS

Solution	When to use
NFS	Simple, existing NFS, dev/small prod
Rook/Ceph	Distributed, production, self-hosted clouds
Longhorn (Rancher)	Cloud-native, lightweight, backup included
OpenEBS	Storage for stateful workloads

Cloud (EBS, PD, Azure Disk)	Managed clouds

Part III

Practical Skills

Chapter 12

kubectl Mastery

kubectl = sayf el DevOps. Ken tśílḥa, tśil 70% mel K8s.

12.1 Essential Commands

Command	Description
`get`	List resources
`describe`	Detailed info + events
`logs`	Container logs
`exec`	Execute command inside container
`apply`	Create or update from YAML
`delete`	Remove resource
`scale`	Change replicas count
`rollout`	Manage deployments (status, history, undo)
`port-forward`	Forward local port to pod
`cp`	Copy files to/from container
`top`	Show resource usage (needs metrics-server)
`edit`	Edit resource in place ($EDITOR)
`config`	Manage kubeconfig contexts

12.2 Flags Critiques

Flag	Description
`-n <ns>`	Namespace
`-A, --all-namespaces`	Kol el namespaces
`-o yaml`	Output YAML
`-o json`	Output JSON
`-o wide`	More columns (IP, node, age)
`-o jsonpath={...}`	Extract field
`-l <selector>`	Label selector (`app=nginx`)

`-w`	Watch for changes
`-f <file>`	From file
`--dry-run=client`	Test without applying
`--previous`	Previous container (crashes)

12.3 Power Commands

Listing 12.1: Commands kol DevOps lezmou y3rafhom

```
# Pod IPs + nodes
kubectl get pods -o wide

# Events (m\'ertabin bel time)
kubectl get events --sort-by=.metadata.creationTimestamp

# Resource usage
kubectl top nodes
kubectl top pods -A

# Delete pod (Deployment y3awd\'ou)
kubectl delete pod my-nginx-abc123

# Restart deployment (ma fammach restart mais...)
kubectl rollout restart deployment/my-nginx

# Port-forward (access service locally)
kubectl port-forward svc/my-service 8080:80

# Copy files
kubectl cp my-pod:/var/log/app.log ./app.log

# Exec interactive shell
kubectl exec -it my-pod -- /bin/bash

# Debug: ephemeral container (K8s 1.25+)
kubectl debug my-pod -it --image=busybox

# Generate YAML without applying
kubectl create deployment nginx --image=nginx --dry-run=client -o yaml

# Apply + save config
kubectl apply -f manifest.yaml --record

# Diff avant apply
kubectl diff -f manifest.yaml
```

12.4 kubectl Aliases (Productivity)

Listing 12.2: Zid fi .bashrc / .zshrc

```
alias k='kubectl'
alias kgp='kubectl get pods'
alias kgs='kubectl get services'
alias kgd='kubectl get deployments'
alias kaf='kubectl apply -f'
alias kdel='kubectl delete'
alias klog='kubectl logs'
alias kexec='kubectl exec -it'
alias kctx='kubectl config current-context'
alias kns='kubectl config set-context --current --namespace'

# Enable autocompletion
source <(kubectl completion bash)
complete -F __start_kubectl k
```

12.5 Debugging Techniques

12.5.1 Pod Stuck en Pending

```
kubectl describe pod <name>
# Lawej fi "Events" section:
#  - Insufficient cpu/memory → node ma y\'k\'afich
#  - No nodes available → nodeSelector/taints problem
#  - FailedScheduling → ressources k'afr\'ikin
```

12.5.2 CrashLoopBackOff

```
# Logs mel container el akher crashi
kubectl logs <pod> --previous

# Check exit code
kubectl describe pod <pod>

# Common causes:
# - Config manquante
# - Liveness probe yet\'en5as
# - Missing dependency (DB down)
# - Application bug
```

12.5.3 ImagePullBackOff

```
kubectl describe pod <pod>
# Events:
# - "manifest unknown" → image tag incorrect
# - "unauthorized" → lezem imagePullSecret
# - "no such host" → network / DNS problem
```

Chapter 13

YAML Manifests Mastery

13.1 Structure Standard

```yaml
apiVersion: <group/version>   # e.g., apps/v1, v1, networking.k8s.io/v1
kind: <ResourceType>          # Pod, Deployment, Service, etc.
metadata:
  name: <name>                # Unique per namespace
  namespace: <ns>             # Optional, default "default"
  labels:                     # Key-value pairs for selection
    key: value
  annotations:                # Key-value pairs for metadata
    key: value
spec:
  # Resource-specific configuration
```

13.2 Best Practices

1. **Toujours est3mel Deployment, mouch Pod tout seul.**
2. **Define resource requests + limits.** Sans, scheduling random, evictions surprises.
3. **Use labels consistently.** `app`, `version`, `env`, `tier`.
4. **Separate configs m3a ConfigMap + Secret.** Ma tḥardćódi.
5. **Readiness + Liveness probes.** Essentials for zero-downtime deploys.
6. **Use namespaces.** Separate environments, teams.
7. **Version your manifests.** Git repo, review via PRs (GitOps).
8. **Use Helm wla Kustomize.** Ma tcopierch nafs manifest 10 marra.

13.3 Probes — Health Checks

```yaml
containers:
- name: app
  image: myapp:v1
```

```yaml
    livenessProbe:           # Ken yeche, restart container
      httpGet:
        path: /healthz
        port: 8080
      initialDelaySeconds: 30
      periodSeconds: 10
    readinessProbe:          # Ken yeche, ra7el mel service endpoints
      httpGet:
        path: /ready
        port: 8080
      initialDelaySeconds: 5
      periodSeconds: 5
    startupProbe:            # Lel apps qui 5adhen wa9t tebdet
      httpGet:
        path: /startup
        port: 8080
      failureThreshold: 30
      periodSeconds: 10
```

Tip Pratique

Rule of thumb lel probes:

- `startupProbe`: lel slow-start apps (JVMs, Rails).
- `readinessProbe`: Check ken ready yakoul traffic.
- `livenessProbe`: Check ken mazal alive (ken dead → restart).

Ma t5attich livenessProbe aggressive ken app mte3ek slow-start!

Chapter 14

Troubleshooting Guide

14.1 Node Issues

14.1.1 Node NotReady

```
# Step 1: Status
kubectl get nodes
kubectl describe node <name>

# Step 2: kubelet 3al node
vagrant ssh <node>
sudo systemctl status kubelet
sudo journalctl -u kubelet -f

# Common causes:
# - kubelet qaf\'ech → restart
# - containerd down → sudo systemctl restart containerd
# - Disk pressure → df -h, clean images
# - Memory pressure → check workloads
# - Network: can't reach API server → check routing
```

14.1.2 Disk Pressure

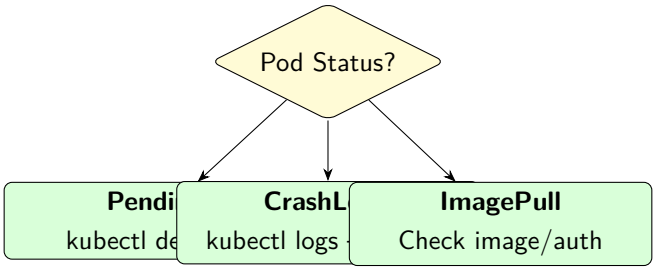
```
# Check disk
df -h

# Clean unused images (3al node)
sudo crictl rmi --prune

# Clean pod logs
sudo find /var/log/pods -type f -size +100M
```

14.2 Pod Issues

14.2.1 Decision Tree



14.3 Network Issues

14.3.1 DNS Ma Y5adchi

```
# Test dns mel pod
kubectl run -it --rm debug --image=busybox --restart=Never -- sh
/ # nslookup kubernetes
/ # nslookup my-service.default

# Check coredns pods
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl logs -n kube-system -l k8s-app=kube-dns
```

14.3.2 Service Ma Yo7ouzch Traffic

```
# Check endpoints (pods match\'es bel selector)
kubectl get endpoints my-service
# Ken vide → selector yenfa ala pods

# Check pods labels
kubectl get pods --show-labels

# Test mel pod debug
kubectl run -it --rm debug --image=nicolaka/netshoot --restart=Never -- \
  curl http://my-service.default.svc.cluster.local
```

14.3.3 Cross-node Communication (CNI)

```
# Flannel pods healthy?
kubectl get pods -n kube-flannel

# Logs
kubectl logs -n kube-flannel -l app=flannel

# Test pod-to-pod cross node
```

```
# Pod A on node1 → ping Pod B on node2 IP
```

14.4 Cluster-Wide Debugging

```
# Health of control plane
kubectl get pods -n kube-system
kubectl get componentstatuses  # deprecated b K8s 1.19+

# Cluster info
kubectl cluster-info
kubectl cluster-info dump > cluster-debug.log

# Events globaux (toujours critical!)
kubectl get events -A --sort-by=.metadata.creationTimestamp | tail -30
```

Chapter 15

Production Readiness

15.1 High Availability

15.1.1 Control Plane HA

- **3 (or 5) master nodes.** Odd numbers (quorum).
- **External etcd cluster** wla **stacked etcd**.
- **Load balancer** lel API server (HAProxy, keepalived, cloud LB).

15.1.2 Application HA

- `replicas >= 2`
- `PodDisruptionBudget` (ma ta7atch all pods down at once)
- `topologySpreadConstraints` (spread across zones/nodes)

Listing 15.1: PodDisruptionBudget

```yaml
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata:
  name: my-app-pdb
spec:
  minAvailable: 2
  selector:
    matchLabels:
      app: my-app
```

15.2 Backup & Disaster Recovery

15.2.1 etcd Snapshots

```bash
# Manual snapshot
sudo ETCDCTL_API=3 etcdctl \
  --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
```

```
    --cert=/etc/kubernetes/pki/etcd/server.crt \
    --key=/etc/kubernetes/pki/etcd/server.key \
    snapshot save /backup/etcd-$(date +%F).db
```

Automate via CronJob, backup lŚ3/NFS.

15.2.2 Velero — Cluster Backup

[

Velero] **Chnoua:** Backup + restore tool lel Kubernetes clusters. Backup PVs (b snapshots) + resources (YAML).

Use cases: Migration, DR, schedule backups.

15.2.3 GitOps — Declarative Recovery

Ken kol manifests fel Git, tnajem ta3mel cluster jdid + `kubectl apply -f` = restore. Tools: ArgoCD, Flux.

15.3 Security Hardening

- **RBAC enabled** (default b kubeadm).
- **Network Policies** (default deny + explicit allow).
- **Pod Security Standards** (restricted profile).
- **Secrets encryption at rest** (etcd).
- **TLS everywhere** (API server, etcd, kubelet).
- **Regular updates** (CVEs patchouhom féssa3).
- **Image scanning** (Trivy, Clair, Grype).
- **Admission controllers** (OPA Gatekeeper, Kyverno).

15.4 Monitoring & Alerting

Stack minimum viable:

- Prometheus (metrics)
- Grafana (dashboards)
- Alertmanager (alerts)
- Loki (logs)
- kube-state-metrics (cluster metrics)
- node-exporter (node metrics)

Alerts critiques:

- Node NotReady > 5 min
- Pod restart rate > 5/hour
- API server latency > 1s
- etcd disk full > 80%
- Certificate expiration < 30 days

Part IV

Next Steps

Chapter 16

Learning Path

16.1 Roadmap lel 6 Mois

1. **Semaine 1-2:** Master kubectl, YAML manifests (hedha ktib).
2. **Semaine 3-4:** Deploy real applications (nginx, WordPress, MySQL).
3. **Mois 2:** Helm (package manager).
4. **Mois 2-3:** CI/CD m3a Jenkins/GitLab/ArgoCD.
5. **Mois 3-4:** Monitoring stack (Prometheus + Grafana).
6. **Mois 4-5:** Security (RBAC, Network Policies, OPA).
7. **Mois 5-6:** Cloud K8s (EKS/GKE) + service mesh (Istio).

16.2 Certifications

Cert	Level	Focus
CKAD	Developer	Apps, deployments, services
CKA	Admin	Cluster mgmt, networking, storage
CKS	Security	Hardening, RBAC, policies

Tips:

- Practice 3la killer.sh (simulator officiel).
- Timing est key — 2 heures lel 17-20 questions.
- `alias k=kubectl` mal awal 7aja!

Chapter 17

Hands-on Exercises

Lou ken téslí7t ktib heda lelkólou, essai les exercices hedhom 3al cluster mte3ek:

17.1 Level 1: Basics

1. Deploy nginx b 3 replicas.
2. Expose via NodePort service.
3. Access mel browser 3al `http://192.168.56.10:<nodeport>`.
4. Scale l5 replicas.
5. Watch pods (`kubectl get pods -w`) w delete pod we7ed — observe comment K8s recreate.

17.2 Level 2: Intermediate

1. Rolling update: nginx:1.25 → nginx:1.26 b zero downtime.
2. Rollback awel version.
3. Create ConfigMap + inject as env vars.
4. Create Secret (base64) + mount as file.
5. Add persistent volume m3a NFS.
6. Resource requests + limits + observe scheduling.

17.3 Level 3: Advanced

1. Deploy multi-tier app (frontend + backend + MySQL).
2. Configure liveness + readiness probes.
3. Setup Ingress controller (nginx-ingress).
4. Install Helm + deploy WordPress via chart.
5. Install Prometheus + Grafana (kube-prometheus-stack).
6. Write CronJob lďatabase backup dawri.

17.4 Level 4: Expert

1. Setup CI/CD: Gitea push → Jenkins build → Nexus push → kubectl apply.
2. Implement NetworkPolicies (switch lĆalico).
3. Enable Pod Security Standards.
4. Setup ArgoCD + GitOps workflow.
5. Deploy Istio service mesh.
6. Disaster recovery: backup etcd, destroy cluster, restore.

Chapter 18

Resources

18.1 Official

- kubernetes.io/docs — *Bible officielle*.
- kubectl Cheatsheet.
- CNCF Landscape — Chouf ecosystem kolou.

18.2 Books

- *Kubernetes in Action* — Marko Lukša (*classic*).
- *Kubernetes Up & Running* — Hightower, Burns, Beda.
- *Kubernetes Patterns* — Ibryam, Huss.
- *Programming Kubernetes* — Hausenblas (pour Operators).

18.3 Free Learning

- Kubernetes the Hard Way — Kelsey Hightower.
- Katacoda (archived but useful).
- Killercoda (interactive scenarios).
- Play with Kubernetes.

18.4 Communities

- Kubernetes Slack (70k+ members).
- r/kubernetes.
- discuss.kubernetes.io.

18.5 Tools tḱsebha

- `k9s` — Terminal UI pour K8s (*yfou9ek klal*).
- `stern` — Multi-pod log tailing.
- `kubectx` + `kubens` — Switch contexts/namespaces féssa3.
- `lens` — GUI pour K8s.
- `kubeseal` — Sealed Secrets CLI.
- `helm` — Package manager.
- `skaffold` — Dev workflow.
- `kustomize` — Config management (built into kubectl).

Conclusion

Ya sa7bi, ken wselt el ghaya mel ktib heda, *chapo!* Mush kól 7ad yfout fi 150+ page mta3 Kubernetes.

El 7a9i9a: **learning Kubernetes mouch destination, houa voyage.** Ecosystem yetwalled kol jóum. Operators jdod, CNIs jdud, patterns jdud.

Mazéit:

1. *Efhem avant ma t7fdh.* Concepts > commands.
2. *Pratique kol yóum.* Deploy 7aja, 7atta ken nginx.
3. *Read post-mortems.* Ki cluster y5asser 3ánd 5arrńnín, efhem el root cause.
4. *Join community.* Slack, Reddit, meetups.
5. *Contribuer.* Documentation, bug reports, meme ken 7aja 9ásira.

Rappel:

> "In production, complexity is the enemy."
> — Kelsey Hightower

Kólma tśi kubernetes deployment complex, rod royeḱ. Fih solution m basset?
Success fi rihletek!

— El Ktib Kemel —

Lou ken sebit fi errors wla 7abb tźid 7ajjat, open PR 3al repo.